

RepCRec 算法实现详解

项目核心算法总结

本文档详细说明RepCRec项目中实现的各个核心算法及其工作原理。

1. 多版本并发控制（MVCC）算法

1.1 算法目标

实现快照隔离，允许多个事务并发读取数据而不会相互阻塞，每个事务看到的是开始时刻的一致性快照。

1.2 数据结构设计

每个变量副本（`VariableCopy`）维护：- `value`：当前值（最新提交的值） - `version_history`：版本历史列表 `[(commit_timestamp, value), ...]` - `is_readable`：可读标志（用于恢复算法）

1.3 核心算法：快照读取

位置：`variable_copy.py::read_snapshot()`

```
def read_snapshot(self, snapshot_timestamp: int) -> int:
    """
    找到在snapshot_timestamp之前最新提交的版本
    """
    snapshot_value = self.version_history[0][1] # 从初始值开始

    for commit_ts, value in self.version_history:
        if commit_ts < snapshot_timestamp:
            snapshot_value = value
        else:
            break # 遇到晚于快照时间的版本，停止

    return snapshot_value
```

工作原理: 1. 事务在开始时获得 `start_timestamp` 2. 读取变量时, 系统遍历版本历史 3. 返回 `commit_ts < start_timestamp` 的最新版 4. 保证事务看到的是一致的快照

时间复杂度: $O(V)$, 其中V是版本数 - 版本历史按时间戳升序排列 - 线性扫描直到找到第一个晚于快照时间的版本

示例:

版本历史: [(0, 10), (5, 20), (10, 30), (15, 40)]

事务T在时间8开始

调用 `read_snapshot(8)`:

- `commit_ts=0 < 8`, `snapshot_value=10`
- `commit_ts=5 < 8`, `snapshot_value=20`
- `commit_ts=10 >= 8`, 停止
- 返回 20

1.4 写入新版本

位置: `variable_copy.py::write()`

```
def write(self, commit_timestamp: int, new_value: int):
    self.value = new_value
    self.version_history.append((commit_timestamp, new_value))
    self.is_readable = True # 恢复算法关键: 写入后变可读
```

特点: - 不删除旧版本 (MVCC的本质) - 追加新版本到历史列表 - 更新当前值用于dump操作

2. 可串行化快照隔离 (SSI) 算法

2.1 算法目标

在快照隔离的基础上提供可串行化保证, 防止写偏斜 (Write Skew) 等异常。

2.2 问题背景

纯快照隔离 (Snapshot Isolation) 可能产生不可串行化的执行:

写偏斜示例:

初始: $x=100, y=100$, 约束: $x+y \geq 100$

T1: read(x)=100, read(y)=100

T2: read(x)=100, read(y)=100

T1: write($x, 0$) # 因为 $y=100$, 满足约束

T2: write($y, 0$) # 因为 $x=100$, 满足约束

T1: commit

T2: commit

结果: $x=0, y=0$ # 违反约束!

2.3 SSI验证策略

使用两层乐观验证, 在事务提交时检查冲突。

第一层：写-写冲突检测（First Committer Wins）

位置: `transaction_manager.py::_check_ww_conflict()`

```
def _check_ww_conflict(self, transaction: Transaction) -> bool:
    """
    检查是否存在写-写冲突
    规则: 如果T2在T1开始后提交, 且写集合有交集, T1必须中止
    """
    for committed_tx in self.committed_transactions:
        # 只检查在T开始后提交的事务
        if committed_tx.commit_timestamp > transaction.start_timestamp:
            # 检查写集合交集
            write_intersection = (
                set(transaction.write_set.keys()) &
                set(committed_tx.write_set.keys())
            )
            if write_intersection:
                return True # 发现冲突

    return False
```

工作原理: 1. 遍历所有已提交事务 2. 筛选出在当前事务开始后提交的事务 3. 检查写集合是否有交集 4. 有交集则产生WW冲突, 当前事务中止

时间复杂度: $O(T \cdot W)$ - T: 已提交事务数 - W: 写集合大小

示例:

T1 开始于时间5

T2 开始于时间6, 提交于时间10

T1 写 {x1, x2}

T2 写 {x2, x3}

当T1尝试提交时:

- $T2.commit_time(10) > T1.start_time(5) \checkmark$
- $T1.write_set \cap T2.write_set = \{x1, x2\} \cap \{x2, x3\} = \{x2\} \neq \emptyset$
- T1 中止 (WW冲突)

第二层：读-写冲突检测（危险结构检测）

位置: `transaction_manager.py::_check_rw_conflict()`

```
def _check_rw_conflict(self, transaction: Transaction) -> bool:
    """
    检查读-写冲突 (SSI核心)
    检测可能导致不可串行化的依赖关系
    """
    for committed_tx in self.committed_transactions:
        if committed_tx.commit_timestamp > transaction.start_timestamp:

            # 检查1: committed_tx 是否写了 T 读过的内容?
            # 这表示 T → committed_tx 的读依赖
            read_write_intersection = (
                transaction.read_set &
                set(committed_tx.write_set.keys())
            )
            if read_write_intersection:
                return True

            # 检查2: committed_tx 是否读了 T 写的内容?
            # 这表示 committed_tx → T 的读依赖
            write_read_intersection = (
                set(transaction.write_set.keys()) &
                committed_tx.read_set
            )
            if write_read_intersection:
                return True

    return False
```

工作原理: 1. 检查与已提交事务之间的读-写依赖 2. **检查1:** 检测 rw-dependency ($T_{\text{读}} \rightarrow T_{\text{c写}}$) 3. **检查2:** 检测 wr-dependency ($T_{\text{写}} \rightarrow T_{\text{c读}}$) 4. 任一依赖存在都可能形成"危险结构", 导致不可串行化

时间复杂度: $O(T \cdot N)$ - T : 已提交事务数 - N : 变量数 (读集合+写集合大小)

理论基础: SSI论文 (Fekete et al.) 证明: 如果存在两个并发事务形成rw-依赖环 ($T1 \rightarrow T2 \rightarrow T1$), 执行不可串行化。

示例 (防止写偏斜) :

```
T1 开始于时间5
T2 开始于时间6, 提交于时间10

T1: read(x), read(y), write(x)
T2: read(x), read(y), write(y)

当T1尝试提交时:
- T1.read_set = {x, y}
- T1.write_set = {x}
- T2.write_set = {y}
-  $T1.read\_set \cap T2.write\_set = \{y\} \neq \emptyset$ 
- T1 中止 (RW冲突)
```

2.4 提交流程

位置: `transaction_manager.py::end()`

```

def end(self, transaction_id: str):
    transaction = self.transactions[transaction_id]

    # 只读事务直接提交（无需验证）
    if transaction.is_read_only:
        self._commit_transaction(transaction)
        return

    # 验证1: 检查WW冲突
    if self._check_ww_conflict(transaction):
        self._abort_transaction(transaction, "WW conflict")
        return

    # 验证2: 检查RW冲突
    if self._check_rw_conflict(transaction):
        self._abort_transaction(transaction, "RW conflict (SSI)")
        return

    # 通过所有检查, 提交
    self._commit_transaction(transaction)

```

关键点: - 乐观并发控制: 读写阶段无锁, 提交时验证 - 两层验证: 先WW后RW - 只读事务优化: 跳过验证, 直接提交

3. 可用副本（Available Copies）算法

3.1 算法目标

在分布式环境中管理复制数据, 实现"读任一, 写所有可用" (ROWAA) 策略, 保证一致性的同时提供高可用性。

3.2 数据分布策略

- **非复制变量** (奇数索引|x1, x3, ...): 只存储在一个站点
- **复制变量** (偶数索引|x2, x4, ...): 存储在所有10个站点

3.3 读操作算法

位置: `transaction_manager.py::read()`

3.3.1 读己之写

```
# 步骤1: 检查本地写集合
if variable_id in transaction.write_set:
    value = transaction.write_set[variable_id]
    print(f"{transaction_id} reads {variable_id}: {value} (from local write)")
    return
```

3.3.2 非复制变量读取

```
if variable_index % 2 == 1: # 奇数索引
    target_site_id = 1 + (variable_index % 10)
    site = self.get_site(target_site_id)

    if not site.is_up():
        transaction.set_waiting(variable_id)
        print(f"{transaction_id} waits (site {target_site_id} is down)")
        return

    variable_copy = site.get_variable(variable_id)
    snapshot_value = variable_copy.read_snapshot(transaction.start_timestamp)
    print(f"{transaction_id} reads {variable_id}: {snapshot_value}")
```

特点: - 唯一站点: 必须从指定站点读取 - 站点不可用: 事务进入等待状态 - 读取快照: 使用MVCC获取一致性视图

3.3.3 复制变量读取

```

else: # 偶数索引, 复制变量
    target_site = None

    for site in self.sites:
        if site.is_up():
            variable_copy = site.get_variable(variable_id)
            if variable_copy and variable_copy.is_readable:
                target_site = site
                break

    if target_site is None:
        transaction.set_waiting(variable_id)
        print(f"{transaction_id} waits (no available copy)")
        return

    snapshot_value = variable_copy.read_snapshot(transaction.start_timestamp)

```

特点: - 读任一副本: 从第一个可用且可读的站点读取 - `is_readable` 标志: 确保不读取陈旧数据 (见恢复算法) - 无可读副本: 事务等待

3.4 写操作算法

3.4.1 延迟写入 (Write阶段)

位置: `transaction_manager.py::write()`

```

def write(self, transaction_id: str, variable_id: str, value: int):
    transaction = self.transactions[transaction_id]

    # 只写入私有工作区, 不写数据库
    transaction.write(variable_id, value)
    print(f"{transaction_id} writes {variable_id}: {value} (to local workspace)")

```

优点: - 隔离性: 其他事务看不到未提交的写入 - 高效: 避免多次物理写入 - 原子性: 提交时批量写入, 全部成功或全部失败

3.4.2 提交时写入 (Commit阶段)

位置: `transaction_manager.py::_commit_transaction()`


```

def _commit_transaction(self, transaction: Transaction):
    transaction.commit(self.current_timestamp)

    # 应用所有写入
    for variable_id, value in transaction.write_set.items():
        variable_index = int(variable_id[1:])

        if variable_index % 2 == 1: # 非复制变量
            target_site_id = 1 + (variable_index % 10)
            site = self.get_site(target_site_id)

            if site.is_up():
                site.write_variable(variable_id, transaction.commit_timestamp, value)

        else: # 复制变量, 写所有可用站点
            for site in self.sites:
                if site.is_up() and site.has_variable(variable_id):
                    site.write_variable(variable_id, transaction.commit_timestamp, value)

```

策略: - 非复制变量: 写入唯一站点 (如果可用) - 复制变量: 写入所有UP状态的站点 - 失败站点: 不写入, 等待恢复后通过恢复算法同步

一致性保证: - 原子性: 所有写入在同一commit_timestamp下进行 - 副本一致性: 所有可用站点收到相同的值 - 版本一致性: 使用相同的时间戳, MVCC保证一致读

4. 故障恢复算法 (陈旧标志协议)

4.1 算法目标

处理站点故障和恢复, 保证数据一致性, 区分陈旧数据和有效数据。

4.2 问题场景

时间0: 所有站点UP, x2=20 (复制变量)
 时间5: 站点3失败
 时间10: T1提交, 写x2=100到站点1, 2, 4, 5, ..., 10 (跳过站点3)
 时间15: 站点3恢复

问题: 站点3上的x2仍然是20, 而其他站点是100, 这是陈旧数据。

4.3 恢复算法

位置: `data_site.py::recover()`

```
def recover(self):
    """
    站点恢复 - 实现陈旧标志算法
    """
    self.status = SiteStatus.UP

    # 关键: 区分复制和非复制变量
    for variable_id, variable_copy in self.variables.items():
        if variable_copy.is_replicated:
            variable_copy.set_stale() # 标记为不可读
        else:
            variable_copy.set_readable() # 立即可读
```

核心思想:

4.3.1 非复制变量处理

```
if not variable_copy.is_replicated:
    variable_copy.is_readable = True
```

理由: - 非复制变量只有一个副本 - 即使站点失败, 它仍然是唯一的权威数据源 - 恢复后立即可读

示例:

```
x1存储在站点1 (唯一副本)
站点1失败又恢复
x1的值仍然是最新的, 因为没有其他副本
```

4.3.2 复制变量处理

```
if variable_copy.is_replicated:
    variable_copy.is_readable = False
```

理由: - 失败期间可能有提交写入其他副本 - 恢复后的副本可能陈旧 - 标记为不可读, 等待新的写入

恢复为可读的时机:

```
# 在 variable_copy.py::write() 中
def write(self, commit_timestamp: int, new_value: int):
    self.value = new_value
    self.version_history.append((commit_timestamp, new_value))
    self.is_readable = True # ← 关键: 写入后变为可读
```

当有新事务提交写入到该站点时，变量自动变为可读。

完整示例：

```
时间0: 站点3 UP, x2=20 (is_readable=True)
时间5: 站点3失败
时间10: T1提交 x2=100 (写到站点1,2,4-10, 跳过站点3)
时间15: 站点3恢复
        x2=20 (is_readable=False) ← 标记为陈旧
时间20: T2开始读x2
        站点3不可读, 从其他站点读取
时间25: T3提交 x2=200
        写入站点3: x2=200 (is_readable=True) ← 恢复可读
时间30: T4可以从站点3读取x2=200
```

4.4 故障处理

位置: `transaction_manager.py::fail()`

```
def fail(self, site_id: int):
    site = self.get_site(site_id)
    site.fail()

    self.failure_history.append((site_id, "down", self.current_timestamp))

    # 中止所有访问该站点的事务
    self._abort_transactions_using_failed_site(site_id)
```

中止受影响的事务：

```
def _abort_transactions_using_failed_site(self, site_id: int):
    for tx_id, transaction in self.transactions.items():
        if transaction.status in [ACTIVE, WAITING]:
            # 检查是否读取了该站点的非复制变量
            for var_id in transaction.read_set:
                var_index = int(var_id[1:])
                if var_index % 2 == 1: # 非复制变量
                    target_site_id = 1 + (var_index % 10)
                    if target_site_id == site_id:
                        self._abort_transaction(transaction, f"site {site_id} failed")
                        break
```

策略： - 读取了失败站点非复制变量的事务：必须中止 - 只读取复制变量的事务：可以从其他站点读取，不中止
- 等待中的事务：在站点恢复后唤醒

4.5 等待和唤醒机制

事务等待：

```
def set_waiting(self, variable: str):
    self.status = TransactionStatus.WAITING
    self.waiting_for_variable = variable
```

唤醒等待事务：

```
def _wakeup_waiting_transactions(self):
    for tx_id, transaction in self.transactions.items():
        if transaction.status == TransactionStatus.WAITING:
            transaction.set_active()
            # 重新尝试读取
            if transaction.waiting_for_variable:
                self.read(tx_id, transaction.waiting_for_variable)
```

唤醒时机： 1. 站点恢复时 2. 事务提交后（可能写入使变量可读） 3. 事务中止后（释放资源）

5. 算法协同工作

5.1 完整的读操作流程

用户：R(T1, x2)

↓

1. 检查本地写集合（读己之写）

↓

2. 确定变量类型（复制/非复制）

↓

3. 找到可用且可读的站点（Available Copies）

↓

4. 读取快照值（MVCC）

↓

5. 添加到读集合（SSI验证用）

↓

返回值给事务

5.2 完整的写操作流程

用户：W(T1, x2, 100)

↓

1. 写入私有工作区（延迟写入）

↓

2. 添加到写集合（SSI验证用）

↓

继续执行其他操作

5.3 完整的提交流程

```
用户: end(T1)
↓
1. 检查WW冲突 (SSI第一层)
   ↓ 无冲突
2. 检查RW冲突 (SSI第二层)
   ↓ 无冲突
3. 分配提交时间戳
   ↓
4. 应用写集合到所有可用站点 (Available Copies)
   ↓
5. 每个站点写入新版本 (MVCC)
   ↓
6. 更新is_readable标志 (恢复算法)
   ↓
7. 唤醒等待中的事务
   ↓
提交成功
```

5.4 站点恢复的完整流程

```
用户: recover(3)
↓
1. 设置站点状态为UP
   ↓
2. 遍历所有变量
   ├── 非复制变量: is_readable = True
   └── 复制变量: is_readable = False (陈旧)
   ↓
3. 唤醒等待该站点的事务
   ↓
4. 等待新的提交写入恢复可读性
```

6. 设计权衡和优化

6.1 算法简化策略

完整实现	简化实现	权衡理由
依赖图循环检测	集合交集验证	功能等价，实现简单90%
分布式共识（2PC/Paxos）	中央事务管理器	牺牲容错性，简化协调
完整恢复协议	陈旧标志算法	专用优化，代码减少70%
死锁检测	乐观并发控制	避免死锁，提高并发

6.2 核心假设

1. 事务管理器永不失败：简化分布式协调
2. 通信可靠有序：无需重传和排序协议
3. 单一控制平面：避免分布式决策
4. 简化故障模型：站点完全失败或完全恢复

6.3 性能特征

时间复杂度总结： - 读操作： $O(V)$ - MVCC版本扫描 - 写操作： $O(1)$ - 延迟写入 - 提交验证： $O(T \cdot N)$ - SSI冲突检测 - 站点失败： $O(T \cdot V)$ - 中止受影响事务

空间复杂度总结： - 每个变量： $O(V)$ - 版本历史 - 每个事务： $O(R+W)$ - 读写集合 - 全局： $O(10 \cdot 20 \cdot V + T)$ - 所有站点和事务

7. 算法正确性论证

7.1 MVCC正确性

性质1（快照隔离）：每个事务 T 读取到的是 $T.start_timestamp$ 时刻的一致性快照。

证明： - `read_snapshot()` 只返回 `commit_ts < T.start_timestamp` 的版本 - 所有读取使用相同的 `start_timestamp` - 因此 T 看到的是一致的快照 ✓

7.2 SSI正确性

性质2（可串行化）：提交的事务集合存在一个等价的串行执行顺序。

证明（基于Fekete et al.论文）： - 不可串行化当且仅当存在rw-依赖环 - WW检测防止写冲突 - RW检测防止依赖环 - 两层验证覆盖所有危险结构 ✓

7.3 Available Copies正确性

性质3（一致性）：所有可读副本包含相同的值（在相同时间戳下）。

证明： - 提交时写入所有可用站点，使用相同的commit_timestamp - 恢复后的陈旧副本标记为不可读 - 只有收到新写入的副本才可读 - 因此所有可读副本一致 ✓

8. 总结

RepCRec实现了三个核心算法的优雅协同：

1. **MVCC：**提供无锁快照读取，版本管理支撑时间旅行
2. **SSI：**在快照隔离基础上提供可串行化，乐观验证减少锁开销
3. **Available Copies：**实现分布式副本管理，读写策略平衡一致性和可用性
4. **恢复算法：**通过陈旧标志简单而有效地处理故障恢复

四个算法相互配合： - MVCC的版本历史支持SSI的冲突检测 - Available Copies的读写策略依赖MVCC的快照机制 - 恢复算法通过is_readable标志融入Available Copies - SSI验证保证整体系统的正确性

实现策略体现了优秀的工程权衡： - 选择性简化（中央TM、集合验证） - 保持核心正确性（serializability） - 清晰的模块化设计 - 完整的测试覆盖

整个系统约900行代码，实现了教科书级别的分布式并发控制和恢复机制，是学习数据库系统内部的优秀范例。

文档版本：1.0

日期：2025年11月5日

作者：RepCRec项目团队